



ARQUITETURA DE SISTEMAS

AULA 7 - CAMADAS E PIPELINE



ARQUITETURA EM CAMADAS - VISÃO GERAL

- ❑ Definição: Arquitetura em camadas (layered architecture ou n-tier) divide o sistema em camadas horizontais de responsabilidades distintas (ex.: apresentação, lógica de negócio, dados). É um estilo muito comum e considerado padrão de fato em muitas aplicações corporativas pela sua simplicidade e familiaridade.
- ❑ Monolítica: Geralmente implementada como um monólito único (um deployment unit), onde todas as camadas são empacotadas juntas. Isso torna o design simples, porém implica que pequenas mudanças exijam redeploy do sistema inteiro.

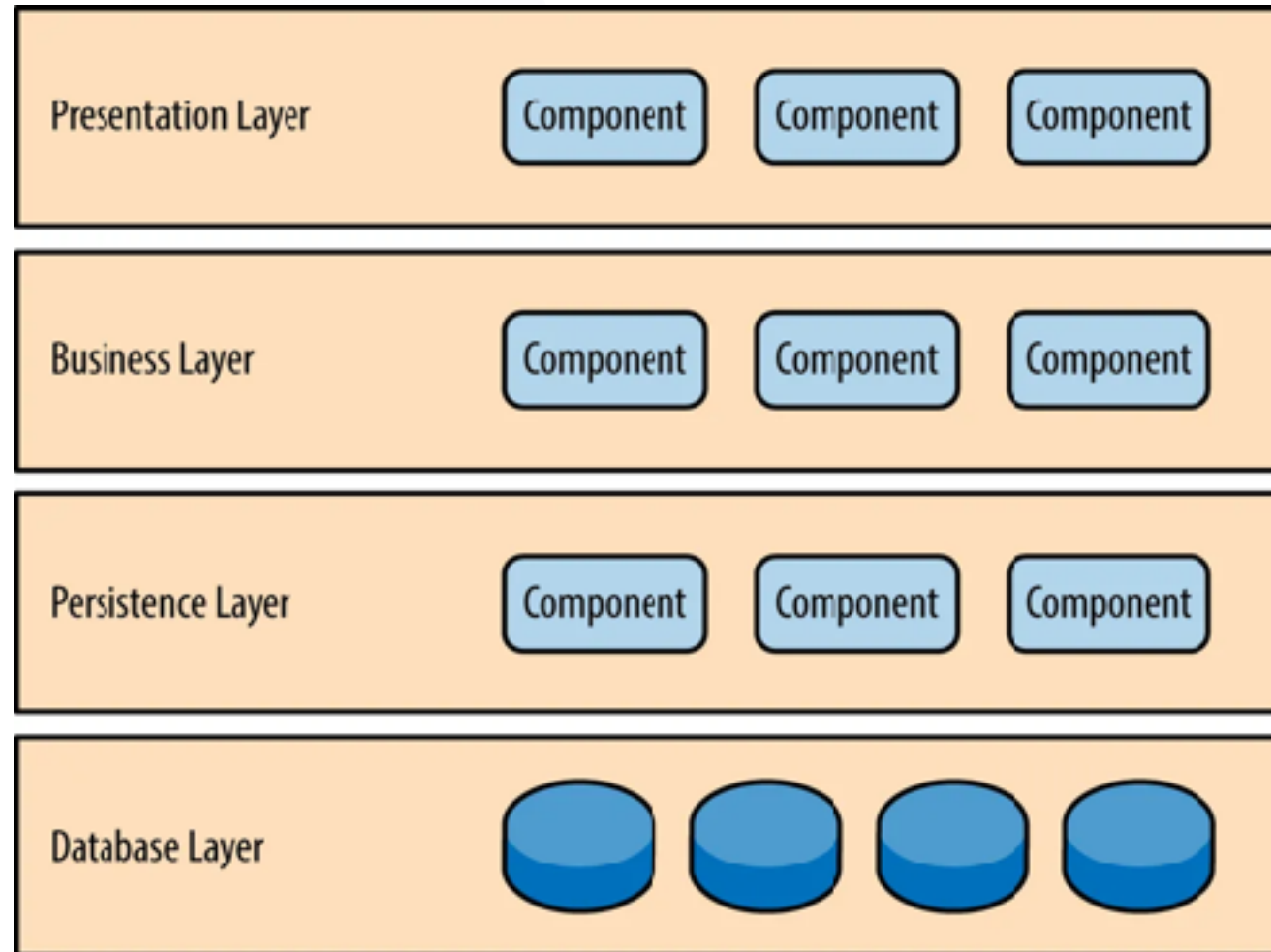


ARQUITETURA EM CAMADAS - VISÃO GERAL

- ❑ Conway's Law: Surge quase “por acidente”. Equipes de front-end, back-end, DB etc. tendem a produzir software estruturado em camadas semelhantes (UI, negócio, BD) naturalmente. Assim, muitos projetos começam em camadas se não houver decisão arquitetural explícita.
- ❑ Anti-padrões: Pode incorrer em anti-patterns como architecture by implication ou accidental architecture, quando se adota camadas sem planejamento formal e acaba-se com um design accidental. Ainda assim, para muitos cenários simples, é um ponto de partida conveniente.



CAMADAS LÓGICAS E RESPONSABILIDADES



Exemplo de arquitetura em camadas com quatro níveis lógicos: Apresentação, Negócio, Persistência e Banco de Dados.

CAMADAS LÓGICAS E RESPONSABILIDADES

- ❑ Camadas padrão: Tipicamente são quatro camadas lógicas
 - ❑ Presentation Layer (camada de apresentação, interface do usuário),
 - ❑ Business Layer (camada de negócio, regras e lógica da aplicação),
 - ❑ Persistence Layer (camada de persistência ou acesso a dados)
 - ❑ Database Layer (camada de dados, banco de dados em si).
- ❑ Cada camada contém componentes responsáveis por aquela função (por exemplo, controllers na apresentação, serviços na negócio, DAOs na persistência, tabelas/SQL no BD).



CAMADAS LÓGICAS E RESPONSABILIDADES

- ❑ Separação de responsabilidades: Cada camada lida apenas com assuntos da sua responsabilidades. A apresentação exibe dados e coleta input do usuário, sem lógica de negócios. A camada de negócio executa regras de negócio usando dados fornecidos, sem se preocupar com UI ou detalhes de armazenamentos. A camada de persistência abstrai operações de BD (queries SQL, ORMs), isolando a lógica de acesso aos dados. Essa divisão clara facilita manutenção e evolução, pois mudanças em uma camada (ex.: trocar o banco) não afetam as outras desde que as interfaces/contratos entre camadas permaneçam estáveis.
- ❑ Variações: Aplicações menores podem combinar camadas (ex.: fundir persistência e negócio em uma única camada de lógica) resultando em 3 camadas. Já sistemas maiores podem adicionar camadas extras, como subdividir a camada de negócio em domain services, application services, etc., ou incluir uma camada de UI web separada de UI desktop. A arquitetura em camadas é flexível quanto ao número de camadas, contanto que cada nova camada tenha um papel bem definido.



REGRAS DE COMUNICAÇÃO E ISOLAMENTO ENTRE CAMADAS

- ❑ Camadas fechadas (closed layers): Por padrão, o estilo em camadas adota isolamento estrito entre níveis. Isso significa que cada camada só pode chamar diretamente a camada imediatamente abaixo (não é permitido “saltar” camadas). Exemplo: a camada de Apresentação chama serviços na camada de Negócio, que por sua vez chama a Persistência, que acessa o Banco de Dados. Essa restrição promove loose coupling, pois uma camada não precisa saber dos detalhes internos das outras, apenas do contrato fornecido. Assim, alterações internas em uma camada (desde que não quebrem a interface) tendem a não impactar as demais (propriedade conhecida como layers of isolation).



REGRAS DE COMUNICAÇÃO E ISOLAMENTO ENTRE CAMADAS

- ❑ Camadas abertas: É possível projetar camadas abertas, onde uma camada superior pode pular intermediárias e acessar camadas mais baixas diretamente. Por exemplo, permitir que a Apresentação consulte o BD para leituras simples, ignorando a camada de Negócio (padrão fast-lane reader já usado em apps no início dos anos 2000). Camadas abertas melhoram performance em certos cenários ao evitar sobrecarga desnecessária, mas quebram o isolamento (mudanças na camada ignorada podem afetar múltiplas camadas de cima, aumentando o acoplamento e fragilidade do sistemas). Em geral, recomenda-se manter camadas fechadas para funcionalidades principais, usando exceções de camada aberta apenas se houver forte justificativa de desempenho.



REGRAS DE COMUNICAÇÃO E ISOLAMENTO ENTRE CAMADAS

- ❑ Anti-pattern Sinkhole: Requisições que percorrem várias camadas apenas repassando dados, sem agregar lógica em cada passo. Ex: a UI solicita dados básicos do cliente; o pedido passa pela camada de Negócio e talvez uma de Regras, que nada fazem além de chamar a Persistência, que finalmente faz a query no BD. O resultado sobe de volta pelas camadas sem processamento adicional. Isso gera overhead de chamadas e objetos sendo criados/destruídos sem necessidade, prejudicando performance. Alguns fluxos “buraco negro” são aceitáveis, mas se a maioria das operações cai nesse caso (e.g. > 80% das requests), é um indicativo de má adequação do estilo em camadas. Nesses casos pode-se flexibilizar camadas (torná-las abertas) ou adotar outro estilo arquitetural.



VANTAGENS DA ARQUITETURA EM CAMADAS

- ❑ Simplicidade: Estilo intuitivo e fácil de entender. As responsabilidades são claras e o fluxo é linear, o que simplifica o design geral. Por não envolver distribuição nem paralelismos complexos, é relativamente barato de construir e manter inicialmente. Equipes costumam ter experiência prévia com camadas, reduzindo a curva de aprendizado.
- ❑ Separação de responsabilidades: Cada camada cuida de um aspecto do sistema (UI, lógica de negócio, acesso a dados, etc.), garantindo organização modular do código. Isso melhora a manutenibilidade, pois alterações em regras de negócio ficam confinadas à camada de negócio, mudanças de BD na camada de dados, e assim por diante. Também possibilita trocar ou refatorar uma camada inteira (por ex.: substituir a tecnologia da interface web) sem grandes impactos nas demais camadas, desde que os contratos sejam mantidos.



VANTAGENS DA ARQUITETURA EM CAMADAS

- ❑ Reutilização: Componentes e lógicas comuns podem ser centralizados em uma camada (ex.: regras de validação na camada de negócio podem ser reutilizadas por diferentes interfaces de apresentação).
- ❑ Especialização: Por exemplo, desenvolvedores front-end trabalham na camada de apresentação e DBAs focam na camada de dados. A estrutura em camadas espelha a divisão de times, facilitando a coordenação do desenvolvimento.
- ❑ Facilidade de teste (em pequena escala): Em projetos simples, pode-se testar cada camada com mocks das camadas adjacentes. Ex.: testar regras de negócio simulando a camada de persistência. Embora a testabilidade global do monólito seja baixa em grandes sistemas, em camadas pequenas permite testes unitários e de integração focados por camada, isolando responsabilidades (cada camada pode ser testada independentemente - até certo ponto).



VANTAGENS DA ARQUITETURA EM CAMADAS

- ❑ Bom ponto de partida: Para aplicações pequenas ou com requisitos pouco claros, a arquitetura em camadas é uma escolha inicial segura. Como é de baixo custo e rápida de implementar, permite entregar valor cedo. Em cenários de prazo curto e equipe reduzida, o monólito em camadas evita a complexidade de arquiteturas mais distribuídas. (Depois, se o sistema crescer, pode-se avaliar refatoração para outro estilo.)



LIMITAÇÕES DA ARQUITETURA EM CAMADAS

- ❑ Desempenho limitado: Sistemas em camadas tendem a ter overhead de chamadas entre camadas, o que impacta a performance em cenários de alta demanda. Como as camadas são geralmente chamadas de forma sequencial e síncrona (não paralela), o throughput fica limitado. O estilo não se presta naturalmente a alta performance ou computação paralela intensiva. Mesmo operações simples podem incorrer no anti-pattern sinkhole (múltiplas camadas fazendo pass-through) adicionando latência desnecessária. É possível mitigar com caching, multithreading dentro do monólito, etc, mas isso exige esforço extra e foge da simplicidade natural do modelo.



LIMITAÇÕES DA ARQUITETURA EM CAMADAS

- ❑ Escalabilidade baixa: Arquiteturas em camadas clássicas são implantadas como um monólito único. Isso dificulta escalar partes específicas do sistema. A única forma de escalar horizontalmente é replicar o sistema inteiro em novos servidores, consumindo mais recursos (não dá para escalar só a camada de negócio, por exemplo, sem escalar também UI e acesso a dados juntos). Embora um monólito possa escalar verticalmente (hardware mais potente) ou replicar instâncias idênticas, a elasticidade e escalabilidade são limitadas a um certo teto, após o qual o sistema torna-se gargalo. Em resumo, aplicações em camadas podem enfrentar dificuldade para atender cargas muito altas ou flutuações abruptas de usuários, a não ser duplicando instâncias inteiras (o que eleva custos e complexidade de sincronização de estado).



LIMITAÇÕES DA ARQUITETURA EM CAMADAS

- ❑ Coupling oculto e complexidade crescente: Embora cada camada seja isolada em teoria, na prática um monólito grande pode desenvolver alto acoplamento interno. Mudanças em uma camada muitas vezes requerem alterações em outra (ex.: modificar um campo de BD afeta persistência e possivelmente lógica). Conforme o sistema cresce, as fronteiras de camadas podem ficar difusas e dependências não planejadas surgem (leaky abstraction). Além disso, um grande módulo monolítico acaba gerando uma base de código pesada (big ball of mud) se não for bem gerenciada, com tempo de build e deploy crescente. Essa complexidade crescente degrada os benefícios iniciais de simplicidade e pode tornar o sistema difícil de modificar ou escalar conforme o número de devs e features aumenta.



OUTRAS DESVANTAGENS DAS CAMADAS

- ❑ Dificuldade de manutenção em larga escala: Em sistemas muito grandes, a separação por camadas técnicas (em vez de por domínios de negócio) espalha funcionalidades de um mesmo domínio em várias camadas. Isso dificulta implementar mudanças de regras de negócio complexas (que podem requerer alterar múltiplas camadas coordenadamente) e prejudica abordagens como Domain-Driven Design, que preferem modularização por funcionalidade de domínio e não por tecnologia. Assim, a agilidade para responder a mudanças de requisitos diminui à medida que o monólito cresce e envolve muitas camadas interdependentes.
- ❑ Deployments lentos e arriscados: Qualquer pequena mudança implica recompilar e fazer o deploy de todo o aplicativo. O processo de implantação costuma ser demorado (parada do sistema, atualização do pacote monolítico, migração de BD, etc.) e de alto risco, pois vários módulos não relacionados podem acabar indo juntos no mesmo deploy. Muitas vezes mudanças são agrupadas em lotes grandes para aproveitar uma janela de deploy, aumentando o risco de algo dar errado. Com isso faz-se deploys menos frequentes e há aumento no tempo de entrega de novas features. Em suma, a pipeline de CI/CD fica lenta em projetos monolíticos gigantes, comparado a arquiteturas de serviços independentes.



OUTRAS DESVANTAGENS DAS CAMADAS

- ❑ Testabilidade reduzida: Testar mudanças pequenas é custoso. Para ter segurança muitas vezes seria necessário rodar um suite de regressão enorme no monólito inteiro, o que nem sempre é viável a cada mudança. Desenvolvedores tendem a não executar todos os testes de ponta a ponta após ajustes triviais, aumentando risco de bugs não detectados. Embora seja possível criar testes unitários simulando outras camadas (com mocks/stubs), testar a integração completa exige levantar toda a stack (banco de dados, servidor app, etc.), o que é complexo. Assim, bugs podem escapar e a confiabilidade geral fica prejudicada.
- ❑ Baixa tolerância a falhas: Uma falha em qualquer parte pode comprometer todo o sistema. Por exemplo, um memory leak ou exceção não tratada em um módulo da camada de negócio pode derrubar a aplicação inteira (processo único). Não há isolamento de processos ou serviços: o sistema em si é um ponto único de falha. Além disso, o MTTR (mean time to recovery) após uma falha tende a ser elevado: reiniciar uma aplicação grande pode levar vários minutos, durante os quais o sistema fica indisponível. Em camadas distribuídas em servidores diferentes (tiers físicos) até se consegue redundância de máquinas, mas dentro do monólito a falha de um módulo ainda derruba a funcionalidade inteira. Comparado a microsserviços, onde uma falha afeta só aquele serviço, no monólito as camadas não protegem contra propagação de falhas.



QUANDO USAR (OU NÃO) ARQUITETURA EM CAMADAS

- ❑ Cenários adequados: Projetos simples ou médios, com lógica de negócio não muito complexa, encaixam bem no estilo em camadas. Se a prioridade é começar rápido e entregar funcionalidades básicas com equipe pequena, o monólito em camadas é apropriado. Também é indicado como ponto de partida quando os requisitos ainda estão sendo entendidos. Inicia-se com camadas pela familiaridade, depois pode-se evoluir a arquitetura se necessário.



QUANDO USAR (OU NÃO) ARQUITETURA EM CAMADAS

- ❑ À medida que a aplicação cresce em tamanho e criticidade, a arquitetura em camadas pode se tornar limitante. Para sistemas que demandam alta escalabilidade (grande volume de usuários ou dados) ou disponibilidade contínua, um monólito grande pode não atender bem. Seria melhor considerar arquiteturas mais particionadas (microsserviços, por exemplo).
- ❑ Se a organização precisa de deploys frequentes e independentes de partes do sistema, o estilo em camadas não atende, pois todas as features estão acopladas no mesmo deploy.
- ❑ Também, se for necessária evolução modular por domínio de negócio (caso de produtos com muitos times atuando em domínios diferentes), a divisão por camadas dificulta. Arquiteturas orientadas a domínio (hexagonal, microkernel, microsserviços) podem ser mais indicadas.
- ❑ Em resumo, use camadas para simplicidade e rapidez inicial; evite-as para sistemas grandes, distribuídos ou de evolução escalável, onde outros estilos oferecem maior modularidade.



ARQUITETURA PIPELINE - VISÃO GERAL

- ❑ Definição: A arquitetura Pipeline (ou Pipes and Filters) organiza o processamento em uma sequência de etapas independentes (filters), conectadas por canais de comunicação (pipes) unidirecionais. Cada filtro realiza uma transformação ou operação nos dados e passa adiante; os pipes conduzem os dados de um filtro ao próximo. É um estilo fundamental muito recorrente. Apareceu historicamente em sistemas de stream (ex.: Unix shell pipelines, onde comandos são ligados através do caracter "|") e em muitos frameworks de processamento de dados.
- ❑ Fluxo unidirecional: Diferente da arquitetura em camadas (que pode ter interações de ida e volta entre UI e BD), no pipeline o modelo típico é one-way flow: dados fluem da origem até um destino final, passando sequencialmente pelos filtros necessários. Não há, em geral, chamada "de volta" de um filtro para o anterior; a comunicação é feed-forward contínua. Isso o torna ideal para processamento de streams, ETL de dados, engines de compilação, etc., onde um conjunto de dados precisa passar por várias transformações em sequência.



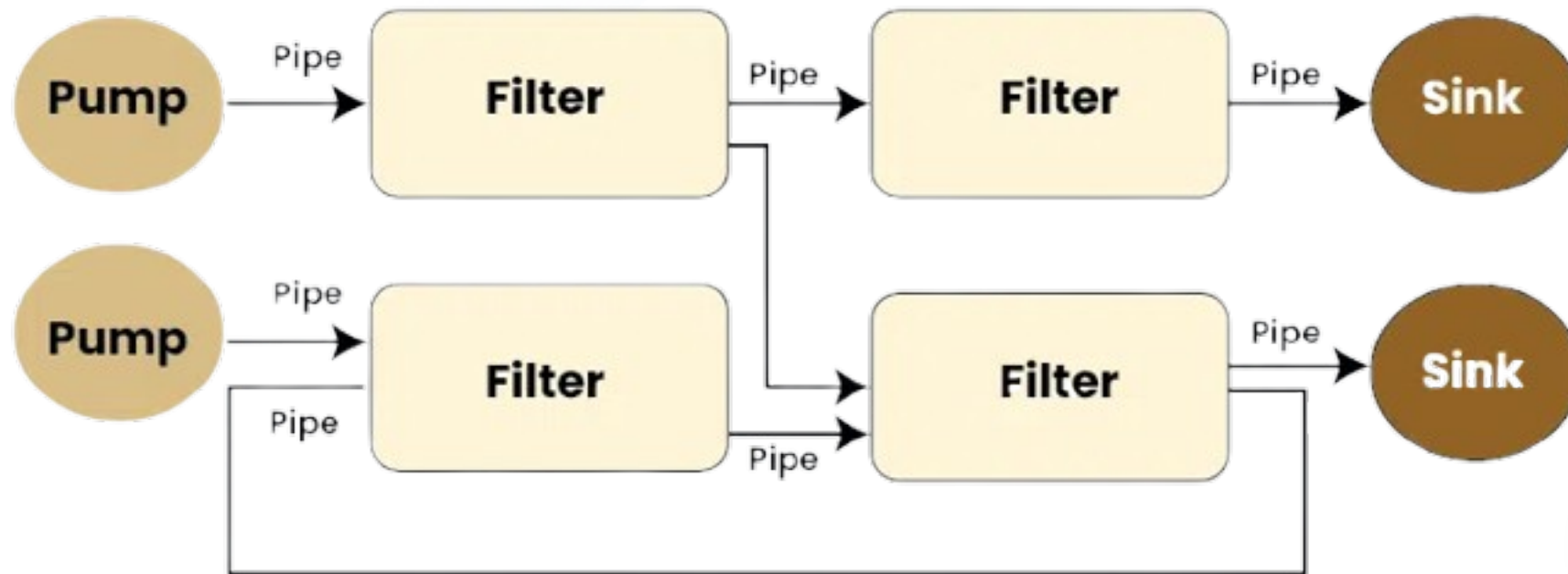
ARQUITETURA PIPELINE - VISÃO GERAL

- ❑ Implementação: Assim como camadas, o pipeline clássico muitas vezes é implementado como um monólito (todos os filtros dentro de um mesmo processo/aplicação). Ainda assim, a separação lógica em etapas facilita entender e modificar o fluxo. Observação: também é possível implementar pipelines distribuídos (cada filtro em um serviço separado conectado por filas), mas aí vira uma arquitetura de microservices integrados por mensagens.
- ❑ Exemplos de uso: Muitas ferramentas de programação funcional e de processamento de dados adotam o padrão pipeline. Por exemplo, o modelo MapReduce divide o processamento em etapas de map e reduce sobre coleções (essencialmente um pipeline). Streams reativos (RX, Java Stream API) também seguem essa ideia de compor operações em sequência. Em integrações, frameworks como Apache Camel permitem definir rotas de processamento (pipeline de filtros) para transformar e rotear mensagens. Assim, o estilo pipeline aparece tanto em implementações de baixo nível quanto em aplicações de negócio que exigem processamento sequencial de informações.



TOPOLOGIA PIPES & FILTERS

Pipe and Filter Architecture - System Design



Esquema de arquitetura Pipeline com dois fluxos paralelos de filtros. Cada fluxo inicia em um Pump (fonte de dados) e termina em um Sink (destino final), passando por filtros conectados por pipes.

TOPOLOGIA PIPES & FILTERS

- ❑ Filters & Pipes
 - ❑ Um Filter é um componente que recebe dados (input), realiza uma operação (p. ex., transformação, validação, cálculo) e emite o resultado no pipe de saída. Os filtros são projetados para serem independentes e geralmente stateless (não guardam estado entre uma mensagem e outra) o que facilita sua composição e reutilização.
 - ❑ Um Pipe é o canal que conecta a saída de um filtro à entrada do próximo. Normalmente é unidirecional e ponto-a-ponto (cada pipe leva a um único próximo filtro, não estilo “broadcast”). Os pipes carregam os dados (podem ser objetos em memória, mensagens, streams bytes etc.), e procura-se manter o formato de dados simples e uniforme para todos os filtros do pipeline, para reduzir acoplamento.



TOPOLOGIA PIPES & FILTERS

- ❑ Categorias de filtros:
 - ❑ Producer: Filtro de início de pipeline, não tem entrada de outro filtro (apenas produz dados iniciais e os coloca no pipe). Ex.: um leitor de arquivo que emite registros para serem processados. (Equivale ao “Pump” mostrado no diagrama acima.)
 - ❑ Transformer: Filtro que recebe input, aplica alguma transformação e emite output modificado. É o tipo mais comum. Cada transformador realiza uma tarefa específica (por exemplo: converter um objeto em JSON, criptografar um campo, calcular um valor agregado).
 - ❑ Tester: Filtro que avalia uma condição sobre os dados e decide encaminhar (ou não) para diferentes saídas. Pode filtrar dados (deixar passar só os que atendem critério) ou rotear para pipelines diferentes. Em programação funcional, seria análogo a um filtro ou parte de um reduce.
 - ❑ Consumer: Filtro final, que consome os dados e termina o pipeline. Pode gravar o resultado em algum lugar (persistir em banco, enviar para outra sistema) ou exibir em tela, etc. (Equivale ao “Sink” no diagrama.)



TOPOLOGIA PIPES & FILTERS

- ❑ Composição e paralelismo: A simplicidade de cada filtro incentiva a composição. Tarefas complexas podem ser alcançadas encadeando filtros simples, ao invés de um componente monolítico fazer tudo. Além disso, se houver múltiplos fluxos independentes de processamento, a arquitetura suporta pipelines paralelos (como mostrado na figura, dois pipelines separados rodando simultaneamente). Cada pipeline trata um subset de dados sem compartilhar estado com o outro, aproveitando multi-threads ou multi-processos para aumento de throughput.

CARACTERÍSTICAS, VANTAGENS E LIMITAÇÕES DO PIPELINE

- ❑ Simplicidade e modularidade: Assim como camadas, o estilo Pipeline preza a simplicidade (cada filtro faz uma coisa específica) e isso traz alta modularidade interna. A divisão clara em etapas permite compreender o fluxo facilmente e modificar ou estender o processamento sem alterar o restante. Por exemplo, adicionar uma nova validação é tão simples quanto inserir um novo filtro no ponto desejado do pipeline.
- ❑ Reuso e manutenção: Filtros bem definidos podem muitas vezes ser reutilizados em diferentes pipelines ou contextos (por ex., um filtro de “converter XML para objeto” pode ser usado em vários fluxos). A independência dos filtros também facilita a manutenção e teste unitário: cada filtro pode ser testado isoladamente fornecendo entradas e verificando saídas esperadas. Alterar a lógica de um filtro (ou substituí-lo por outra implementação) é menos arriscado do que mexer em um módulo monolítico grande, pois o efeito se limita àquela etapa e seus dados de saída.



CARACTERÍSTICAS, VANTAGENS E LIMITAÇÕES DO PIPELINE

- ❑ Desempenho: O pipeline em si realiza processamento sequencial, o que pode não atingir alta performance se uma única sequência tiver que processar volumes gigantes de dados. Não há divisão de trabalho interno automático entre filtros, é passo a passo. Entretanto, por não envolver interações remotas (num cenário monolítico) e cada filtro ser leve, a latência pode ser baixa para fluxos simples. Em contrapartida, é possível mitigar gargalos inserindo múltiplos pipelines paralelos para processar lotes de dados simultaneamente ou usando técnicas de multi-threading dentro de certos filtros, embora isso adicione complexidade.
- ❑ Escalabilidade e Elasticidade: Como o pipeline tradicional é implantado como um único serviço monolítico, ele compartilha limitações de escalabilidade semelhantes às do monólito em camadas. Ou seja, para escalar horizontalmente, tipicamente duplica-se todo o pipeline em outra instância (não dá para escalar “só um filtro” independentemente, pois eles são acoplados no mesmo processo). Isso implica elasticidade baixa (adicionar mais recursos envolve subir mais instâncias monolíticas) e dificuldade em aproveitar computação distribuída sem quebrar o pipeline em partes. É possível adaptar pipelines para distribuição (cada filtro em um container, comunicando via filas), mas aí já se está migrando para outra arquitetura (microsserviços orientados a eventos), fugindo do modelo monolítico original.



CARACTERÍSTICAS, VANTAGENS E LIMITAÇÕES DO PIPELINE

- ❑ Deploy e alterações: Apesar de monolítico, o pipeline tem ligeira vantagem sobre camadas em facilidade de alterações. Como cada filtro é uma unidade relativamente isolada, é teoricamente mais simples desenvolver e implantar uma mudança localizada (por exemplo, modificar um algoritmo em um filtro ou inserir um filtro novo) sem afetar o restante do código. Na prática, porém, qualquer mudança ainda requer redeploy do monólito inteiro (não há separação de pacotes de deploy por filtro), então o ganho é limitado. Deploys frequentes continuam sendo um desafio. Em termos de testes, existe benefício em poder testar filtros individualmente, mas a integração completa do pipeline ainda precisa ser validada de ponta a ponta.
- ❑ Tolerância a falhas: O estilo pipeline não oferece tolerância a falhas elevada quando implementado como monólito. Uma exceção não tratada ou erro crítico em um filtro pode parar todo o pipeline, interrompendo o processamento global. Não há isolamento de falha: se um filtro entra em loop infinito ou consome memória excessiva, todo o aplicativo pode travar, semelhante ao cenário do monólito em camadas. Além disso, a necessidade de reiniciar um pipeline caído traz indisponibilidade proporcional ao tempo de start da aplicação, que em monólitos grandes pode ser significativo. Somente re-arquitetando para filtros isolados (processos separados) conseguiríamos maior resiliência, mas aí vira outra topologia (microservices).



CARACTERÍSTICAS, VANTAGENS E LIMITAÇÕES DO PIPELINE

- ❑ Compatibilidade com microserviços: Tanto a arquitetura em camadas quanto a pipeline, em sua forma monolítica, têm baixa compatibilidade com a abordagem de microserviços. O estilo em camadas encoraja um único sistema contendo todas as funções técnicas, enquanto microserviços preferem divisão por domínios de negócio. Já o pipeline monolítico encadeia etapas internamente; para torná-lo microserviços, cada filtro viraria um serviço independente. O que é possível e às vezes feito (ex.: pipeline de ETL distribuído), mas deixa de ser um monólito e passa a ser uma arquitetura distribuída orientada a eventos.



ATÉ A PRÓXIMA AULA



@eldermoraes